

Amazon ML Challenge

Product Price Prediction: Technical Documentation

Prepared in HTML format for convenient PDF export and professional review.

Table of Contents

- [1. Executive Summary](#)
 - [2. Problem Statement](#)
 - [3. Objectives](#)
 - [4. Solution Overview](#)
 - [5. System Architecture / Pipeline Overview](#)
 - [6. Repository Structure](#)
 - [7. Environment and Installation](#)
 - [8. Data Pipeline](#)
 - [9. Feature Engineering](#)
 - [10. Model Development](#)
 - [11. Evaluation Methodology](#)
 - [12. Benchmarking and Testing Pipeline](#)
 - [13. Training and Usage Guide](#)
 - [14. Prediction / Deployment Workflow](#)
 - [15. Results and Analysis](#)
 - [16. Feature Importance Discussion](#)
 - [17. Limitations](#)
 - [18. Future Improvements](#)
 - [19. Technology Stack](#)
 - [20. Conclusion](#)
 - [21. Acknowledgments](#)
 - [22. Contact Information](#)
-

1. Executive Summary

This document outlines a comprehensive machine learning solution developed for the Amazon ML Challenge. The core task involves predicting product prices utilizing catalog content, product metadata, and computer vision techniques. By applying robust multi-modal feature engineering and ensemble modeling methodologies, the developed system achieves competitive performance. The pipeline effectively parses unstructured text, extracts over 100 distinctive features, processes tens of thousands of product images, and employs advanced gradient boosting and deep learning architectures for precise numerical forecasting.

2. Problem Statement

E-commerce platforms manage vast catalogs of products where pricing can be highly variable and dependent on numerous unstructured attributes. The challenge requires building a machine learning solution capable of analyzing diverse product catalog data—including unstructured text descriptions, structured metadata, and product images—to accurately predict the retail price of items.

Key complexities of this problem include:

- Extracting structured numerical and categorical features from highly unstructured text.
- Accurately parsing quantities, units of measurement, and brand details with high variability in formatting.
- Handling multi-modal datasets combining both textual information and visual elements (images).
- Managing missing, erroneous, or inconsistent data inputs.
- Minimizing the Symmetric Mean Absolute Percentage Error (SMAPE) to ensure robust predictive accuracy across varied price ranges.

3. Objectives

The primary objectives of this project are to:

- Develop an end-to-end data processing pipeline capable of cleaning and standardizing raw catalog data.
- Implement sophisticated Natural Language Processing (NLP) rules and regular expressions (Regex) to extract critical product attributes (brands, object types, pack sizes, quantities).
- Construct an automated, multi-threaded image acquisition and preprocessing system.
- Train, benchmark, and deploy an ensemble of robust regression models capable of delivering low-error price estimations.

4. Solution Overview

The proposed solution utilizes a multi-stage approach, seamlessly integrating data extraction, feature engineering, computer vision preprocessing, and ensemble modeling. Through an automated Continuous Integration/Continuous Deployment (CI/CD) framework structure, the solution parses catalog content dynamically, recognizes over 200 distinct brands, standardizes diverse measurement units, and concurrently evaluates multiple regression models (XGBoost, LightGBM, CatBoost, and PyTorch Neural Networks).

5. System Architecture / Pipeline Overview

The architecture is divided into five primary stages:

- 1. Data Extraction & Parsing:** Ingesting combined train/test sets and applying advanced regex-based functions to separate brands, object types, and item categories.
- 2. Feature Engineering:** Transforming extracted text into standardized, model-ready formats, calculating base quantities (converting solid weights to grams and liquid volumes to milliliters), and generating derived metrics (e.g., price per unit).
- 3. Image Processing:** A concurrent downloading engine fetches product images, followed by standardization and resizing procedures to prepare for future Convolutional Neural Network (CNN) integration.
- 4. Ensemble Modeling:** Training gradient boosting algorithms and deep learning Multi-Layer Perceptrons (MLPs) on the engineered feature space.
- 5. Automated Testing & Deployment:** A dedicated framework to benchmark model outputs, evaluate performance metrics, and automate the deployment of top-performing algorithms.

6. Repository Structure

The project codebase is organized to maintain separation of concerns between raw data, processed outputs, model binaries, scripts, and deployment modules.

```
amazon-ml/
|
├── data/
|   ├── train/                (Contains train.csv, cleaned/parsed variants, images)
|   └── test/                  (Contains test.csv)
|
├── models/
|   ├── xgb.json               (XGBoost model binary)
|   ├── lgbm_model.txt         (LightGBM model)
|   └── model_weights.pth      (PyTorch MLP weights)
|
├── CD_CI/                     (CI/CD module for benchmarking and deployment)
|   ├── ModelBenchmark.py
|   └── automodel_deploy.py
|
```

— eda.py	(Exploratory Data Analysis script)
— extract.py	(Feature extraction pipeline)
— Parser.ipynb	(Catalog content parsing notebook)
— image_downloader.ipynb	(Multi-threaded image downloader)
— image_loader.py	(Image download script)
— image_clean.ipynb	(Image preprocessing)
— Model_Train_0.5.ipynb	(Core model training notebook)
— model.py	(Model architectures definitions)
— TestPipeline.py	(Automated testing pipeline execution)
— X_train.pkl, y_train.pkl	(Processed training matrices)
— X_test.pkl, y_test.pkl	(Processed test matrices)
— requirements.txt	(Python dependencies)
— run_file.bat	(Windows activation script)

7. Environment and Installation

The project requires a Python 3.13+ environment. A Virtual Environment is highly recommended, alongside GPU hardware for optimal PyTorch acceleration.

Installation Steps

```
# 1. Clone the repository
git clone https://github.com/yourusername/Amazon-ML-Challenge.git
cd Amazon-ML-Challenge

# 2. Create and activate virtual environment (Windows)
python -m venv .venv
.venv\Scripts\activate

# 3. Install dependencies
pip install -r requirements.txt

# 4. Verify installation
python model.py
```

Executing `model.py` will output the availability status of the required ML frameworks, confirming a successful setup configuration.

8. Data Pipeline

Data Loading and Combination

Located in `Parser.ipynb`, the data loader concatenates training and testing datasets. Combining these ensures unified preprocessing logic and prevents discrepancies in feature encoding or vocabulary generation between the sets.

Catalog Content Parsing

This stage represents the core parsing engine. It is tasked with extracting structured insights from highly descriptive, unstructured product entries. Using dynamic identification, the parser matches keywords across 32 defined product types (e.g., sauce, cookies, beverage) and handles over 200 known brand names. The system infers missing quantities by utilizing the average quantity specific to an item's product type.

Feature Extraction

The `extract.py` module manages sophisticated extraction logic using multi-pattern regex matching. It effectively isolates brand names from product descriptors while filtering out subjective descriptors like "organic" or "fresh". It possesses robust detection for multi-pack configurations (e.g., "pack of 6", "case of 24").

Image Data Collection

To harness visual data, a multi-threaded downloading script (`image_loader.py`) utilizes a `ThreadPoolExecutor` to fetch images concurrently across 100 worker threads. It incorporates resume capabilities, error logging, rate limiting, and timeout management to prevent server blocking, achieving an average throughput of 14-17 images per second.

Image Preprocessing

Once retrieved, images are standardized utilizing the `image_clean.ipynb` notebook. Utilizing the Pillow (PIL) library, files are resized to a uniform 100x100 resolution using LANCZOS resampling. This methodology mitigates computational overhead for CNN operations while preserving high-quality aspect ratios and edge clarity.

9. Feature Engineering

Feature engineering is segregated into structured domains representing the distinct aspects of the product profile.

- **Text-Based Features:** Categorical identification of `brand_name`, `object_type`, `item_type`, and physical `state` (solid/liquid).
- **Quantity Features:** Standardizing the original quantities (`quantity_value_raw`, `quantity_unit_raw`) into a unified base metric system (`quantity_value_base`). The module accurately translates ounces, pounds, and kilograms into grams, and fluid ounces or liters into milliliters. `total_base_qty` reflects the base unit multiplied by any detected `packs_count`.
- **Derived Metrics:** Analytical signals such as binary indicators (`has_brand`, `has_quantity`), character counts (`item_name_length`), price-to-volume ratios (`price_per_unit`), and normalized scale variables (`log_quantity`).

10. Model Development

XGBoost Regressor

Configured with 200 estimators and a learning rate of 0.1, the XGBoost model serves as the primary gradient boosting baseline. It excels in handling mixed categorical and continuous data types and exhibits high robustness against numerical outliers inherent in retail pricing.

LightGBM Regressor

Utilizing 31 leaves and 200 estimators, LightGBM is incorporated for its operational efficiency. It provides notably faster training intervals and lower memory consumption compared to XGBoost, performing efficiently on high-cardinality categorical variables.

CatBoost Regressor

CatBoost is deployed utilizing a depth configuration of 6. Its inclusion in the ensemble relies heavily on its native support for categorical features without requiring explicit prior label encoding, preventing target leakage and mitigating overfitting risks.

PyTorch MLP

A Multi-Layer Perceptron architecture developed using PyTorch. The network contains consecutive fully connected layers transitioning from the input dimension through 128 nodes, to 32 nodes, and culminating in a singular output node. Gaussian Error Linear Units (GELU) are utilized as activation functions to ensure smoother gradient propagation. This architecture captures complex non-linear feature interactions that tree-based algorithms may overlook.

11. Evaluation Methodology

The primary evaluation metric mandated by the challenge is the Symmetric Mean Absolute Percentage Error (SMAPE).

```
def smape(y_true, y_pred):  
    return np.mean(  
        2.0 * np.abs(y_pred - y_true) /  
        (np.abs(y_true) + np.abs(y_pred) + 1e-12)  
    ) * 100
```

SMAPE is selected because it is symmetric, scale-independent, and bounds errors between 0% and 200%, offering a stabilized and fair comparative baseline across heavily varying price stratifications. To prevent zero-division anomalies, an epsilon of 1e-12 is appended to the denominator. Secondary metrics tracked include MAE, RMSE, and R² Score.

12. Benchmarking and Testing Pipeline

The `TestPipeline.py` leverages a custom CI/CD module (`ModelBenchmark`) to assess model efficacy automatically. The framework loads designated model files, processes inference across testing sets, and outputs aggregated tabular results comparing SMAPE, error distributions, and inference latency, enabling swift iterative improvements.

13. Training and Usage Guide

To replicate the environment and training results, the process must be executed sequentially:

```
# 1. Parse content
jupyter notebook Parser.ipynb

# 2. Extract structured features
python extract.py

# 3. Process Exploratory Data Analysis & Data Serialization
python eda.py

# 4. Train Models
jupyter notebook Model_Train_0.5.ipynb
```

Executing the Test Pipeline script (`TestPipeline.py`) assesses performance outputs globally across all trained instances.

14. Prediction / Deployment Workflow

The framework facilitates easy deployment via the `automodel_deploy.py` utility. Implementing this utility within a Python script allows dynamic execution of predictions on novel catalog additions.

```
from CD_CI.automodel_deploy import load_and_predict
import pandas as pd

# Ingest new product dataset
deployment_df = pd.read_csv("new_products.csv")

# Generate aggregate predictions
predictions = load_and_predict(deployment_df)
print(predictions['xgb_predictions'])
```

15. Results and Analysis

Note on Model Performance Discrepancies

There is an acknowledged discrepancy regarding the baseline XGBoost performance within the initial project summary (citing 61.12% SMAPE) versus the formal benchmarking outputs shown below (citing 69.12% SMAPE). For the purpose of rigorous technical documentation, the validated tabulated benchmarking values (69.12%) are treated as the definitive performance metrics for this iteration of the codebase.

Model Performance Comparison

Model	SMAPE	MAE	RMSE	Average Training Time
XGBoost	69.12%	\$12.45	\$18.32	2 min
LightGBM	70.85%	\$12.89	\$18.67	1 min
CatBoost	71.34%	\$13.21	\$19.12	3 min
PyTorch MLP	71.28%	\$13.12	\$19.01	5 min

Parsing and Data Extraction Statistics

Metric	Count	Percentage
Total Products Processed	150,000	100%
Successfully Parsed Names	145,223	96.8%
Extracted Brands	138,730	92.5%
Extracted Object Types	131,887	87.9%
Extracted Quantities	147,568	98.4%
Images Downloaded	143,250	95.5%

16. Feature Importance Discussion

Interpretability analysis on the XGBoost regressor reveals the leading features driving price prediction outcomes:

- total_base_qty (18.5%):** The definitive quantity scaling metric directly correlates with bulk pricing dynamics.
- brand_name_encoded (12.3%):** Premium and luxury branding highly dictates unit price ceilings.

3. `item_type_encoded` (11.7%): Category variances (e.g., consumer electronics vs. dry goods).
4. `state_encoded` (8.9%): Distinctions in packaging and transportation costs between liquid and solid states.
5. `packs_count` (7.2%): Granular insight into multi-unit wholesale configurations.

17. Limitations

While the current architecture parses text data extensively, it heavily relies on heuristic regex and predefined keyword banks, leading to potential drops in accuracy when processing niche or globally varied catalogs lacking standardized nomenclature. Furthermore, while the image acquisition pipeline is operational, deeply embedded visual features from Convolutional Neural Networks are slated for full integration in a later release.

18. Future Improvements

Immediate roadmap enhancements focus on lowering the SMAPE threshold below 65%.

- **Image Feature Integration:** Implementing pre-trained ResNet50 or EfficientNet models to generate structural embeddings from the processed 100x100 images, capturing package aesthetics and color histograms.
- **Advanced NLP Implementation:** Replacing heuristic text parsing with transformer architectures (BERT embeddings) and TF-IDF matrix generation to attain deeper semantic relationships across product descriptions.
- **Ensemble Stacking:** Utilizing a meta-learner architecture to dynamically weight predictions from the XGBoost, LGBM, and MLP branches.
- **Production Grade Serving:** Containerizing the models via Docker and deploying a RESTful interface using FastAPI for seamless external platform integration.

19. Technology Stack

The solution relies on modern, robust open-source libraries emphasizing performance and scalability:

- **Core Machine Learning:** XGBoost, LightGBM, CatBoost, PyTorch, Scikit-learn.
- **Data Wrangling & Processing:** Pandas, NumPy.
- **Computer Vision & Image Ops:** Pillow (PIL).
- **Pipeline Utilities:** concurrent.futures (Multi-threading), tqdm, joblib, requests.

20. Conclusion

This project provides a substantial foundation for automated, multi-modal price prediction. Through stringent parsing, extensive multi-threaded pipeline engineering, and systematic benchmarking, the proposed solution proves resilient against the vast inconsistencies inherent in unstructured catalog descriptions. With future deep-learning visual integration, predictive performance will scale even further.

21. Acknowledgments

Gratitude is extended to the organizers of the Amazon ML Challenge for defining the rigorous problem space, and the broader open-source Kaggle and machine learning communities for ongoing inspiration in handling intricate data preprocessing challenges.

22. Contact Information

For technical inquiries or potential collaborations regarding this codebase:

- **Email:** omeshmehta70@gmail.com
- **GitHub Repository:** [@Omesh2004](#)

End of Document